

Radio Live Transcription System: Architecture & API Specification (v1.0.0)

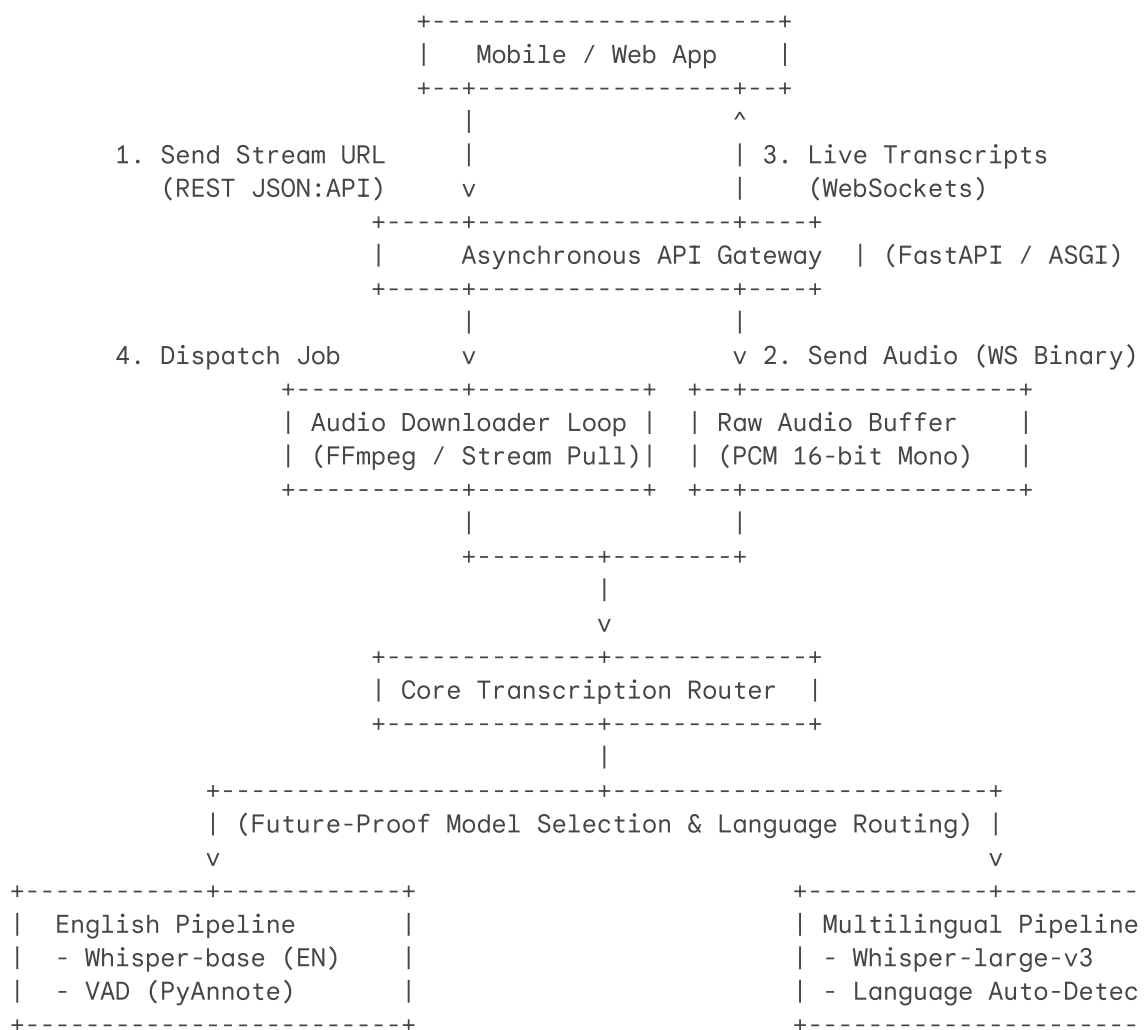
This document provides the formal architectural design and API specification for the **Radio Live Transcription System**.

The system is a Python-based backend that ingests real-time audio (either pulled directly from a streaming radio URL or pushed as raw audio streams from a client mobile app), decodes it, passes it through an extensible Speech-To-Text (STT) pipeline, and delivers live text chunks to clients using standard WebSockets.

1. System Architecture & Multilingual Readiness

The backend is built in **Python** (utilizing an asynchronous framework such as FastAPI/Sanic) to leverage its extensive ecosystem for real-time streaming, high-concurrency WebSockets, and Machine Learning.

1.1 Structural Component Architecture



1.2 Multi-Language Architectural Strategy

To pivot from English-only to a global, multi-language transcription engine smoothly, the architecture separates the **Ingestion Layer** from the **Inference Pipeline**:

1. **Dynamic Language Routing:** Every transcription request contains a `language` parameter within the API request payloads.
 - `"en"` defaults to highly-optimized English-only models (e.g., Whisper English-only variants which are faster and require less VRAM).
 - Other language codes (e.g., `"es"`, `"fr"`, `"zh"`) trigger routes utilizing multilingual model weights (e.g., `Whisper-large-v3` or localized `wav2vec` models).
2. **Language Auto-Detection (LAD):** If `language` is set to `"auto"`, the system routes the first N seconds of incoming audio to a lightweight language classification head (or uses Whisper's native primary-language estimator) to dynamically lock the model settings for that session.
3. **Pluggable Engine Drivers:** The engine uses a unified wrapper interface (`BaseSTTEngine`) making it trivial to plug in external APIs (like Deepgram, AssemblyAI) or local pipelines (like Faster-Whisper, vLLM-Speech) depending on the target language's hardware requirements and accuracy profile.

2. Authentication & Cryptographic Key Protocol

To keep authentication simple, lightweight, and robust, the system operates exclusively using secure **API Keys**.

2.1 API Key Derivation & Storage (Database-side)

API Keys must never be stored in plain text. We utilize a split-key cryptographic model (similar to AWS or Stripe):

- **Key Format:** `rts_live_[PublicID]_[SecretToken]`
 - `PublicID` : A 12-character base62 string used to look up the database record rapidly.
 - `SecretToken` : A 32-character cryptographically secure random string.
- **Storage Encryption:** The server stores only the cryptographic hash of the `SecretToken` using the **Argon2id** password-hashing algorithm, or **SHA-256** if low latency is critical.

$$\text{StorageHash} = \text{SHA-256}(\text{SecretToken} \parallel \text{Salt})$$

2.2 Client-to-Server Verification (HMAC-SHA256 Signature Workflow)

To prevent API key leakage over transit, clients can sign their requests instead of sending the key in plain text (though TLS prevents sniffing, signatures prevent replay attacks).

[Client]		[Server]
-- 1. Generate Request Timestamp (t) ----->		
-- 2. Calculate Signature (s):		
s = HMAC-SHA256(SecretToken, Payload + t)		
-- 3. Send HTTP Request / WS Connection Handshake ----->		
Headers:		
- X-API-Key-ID: rts_live_[PublicID]		
- X-Signature: s		
- X-Timestamp: t		

|
|
|
|
|
|
|
v

| 4. Retrieve
| matching
| 5. Check if
| (e.g., <
| 6. Recomput
| (Time-co
| 7. Authoriz
v

3. Strict JSON:API 1.1 Specification REST API

The system exposes five rest endpoints. All HTTP communication complies strictly with the **JSON:API 1.1 Specification**.

- **Media Type Requirement:** All requests and responses must use the HTTP Content-Type header: `application/vnd.api+json`.
- **Standard Root Document Schema:** Every document must include `jsonapi: { "version": "1.1" }`.

The Five REST Endpoints

1. `POST /api/v1/sessions` - Initiate a server-processed transcription session (pull from a radio stream URL).
2. `POST /api/v1/client-sessions` - Register and provision a client-pushed transcription session (generating credentials/tokens for subsequent WebSocket audio streams).
3. `GET /api/v1/sessions` - Retrieve historical list of transcription sessions with filtering, sorting, and pagination.
4. `GET /api/v1/sessions/{id}` - Fetch details and current status of a specific transcription session.
5. `GET /api/v1/languages` - Query the available systems, models, and supported languages (multilingual system registry).

Endpoint 1: Create Server-Processed Session (Pull Stream)

- **Method:** `POST`
- **Path:** `/api/v1/sessions`
- **Description:** Tells the Python server to directly download and transcribe an external radio URL.

Request Document

```
{
  "jsonapi": {
    "version": "1.1"
  },
  "data": {
    "type": "sessions",
    "attributes": {
      "streamUrl": "https://icecast.radio-station.com/live.mp3",
      "title": "Evening Drive Radio Show",
      "language": "en",
      "model": "high-fidelity",
      "autoRestart": true
    }
  }
}
```

```

    }
  }
}

```

Response Document (201 Created)

```

{
  "jsonapi": {
    "version": "1.1"
  },
  "links": {
    "self": "https://api.transcriptions.io/api/v1/sessions"
  },
  "data": {
    "type": "sessions",
    "id": "sess_89324792184",
    "attributes": {
      "streamUrl": "https://icecast.radio-station.com/live.mp3",
      "title": "Evening Drive Radio Show",
      "language": "en",
      "model": "high-fidelity",
      "status": "processing",
      "createdAt": "2026-06-03T07:15:00Z"
    },
    "relationships": {
      "transcripts": {
        "links": {
          "self": "https://api.transcriptions.io/api/v1/sessions/sess_89324792184",
          "related": "https://api.transcriptions.io/api/v1/sessions/sess_89324792"
        }
      }
    },
    "links": {
      "self": "https://api.transcriptions.io/api/v1/sessions/sess_89324792184"
    }
  }
}

```

Endpoint 2: Provision Client-Pushed Session (Mobile App Pipeline)

- **Method:** POST
- **Path:** /api/v1/client-sessions
- **Description:** Pre-allocates resources for an upcoming client-to-server WebSocket audio stream session. It yields an authenticated endpoint token for WebSocket handshakes.

Request Document

```

{
  "jsonapi": {
    "version": "1.1"
  },
  "data": {
    "type": "client-sessions",

```

```

    "attributes": {
      "deviceName": "iOS iPhone 15 Pro",
      "samplingRate": 16000,
      "audioFormat": "audio/x-raw",
      "language": "en"
    }
  }
}

```

Response Document (201 Created)

```

{
  "jsonapi": {
    "version": "1.1"
  },
  "links": {
    "self": "https://api.transcriptions.io/api/v1/client-sessions"
  },
  "data": {
    "type": "client-sessions",
    "id": "csess_01HJ8Z82K0W",
    "attributes": {
      "deviceName": "iOS iPhone 15 Pro",
      "samplingRate": 16000,
      "audioFormat": "audio/x-raw",
      "language": "en",
      "websocketUrl": "wss://stream.transcriptions.io/v1/stream?session_id=csess_01HJ8Z82K0W",
      "handshakeToken": "hst_98fe78ba8328470a273bf92d"
    },
    "links": {
      "self": "https://api.transcriptions.io/api/v1/client-sessions/csess_01HJ8Z82K0W"
    }
  }
}

```

Endpoint 3: Retrieve Sessions Directory

- **Method:** GET
- **Path:** /api/v1/sessions
- **Description:** Queries a historical list of transcription jobs. Supports JSON:API query standards (filter , sort , page).

Response Document (200 OK)

```

{
  "jsonapi": {
    "version": "1.1"
  },
  "links": {
    "self": "https://api.transcriptions.io/api/v1/sessions?page[size]=1&page[numb",
    "first": "https://api.transcriptions.io/api/v1/sessions?page[size]=1&page[numb",
    "next": "https://api.transcriptions.io/api/v1/sessions?page[size]=1&page[numb",
    "last": "https://api.transcriptions.io/api/v1/sessions?page[size]=1&page[numb"
  }
}

```

```

},
"data": [
  {
    "type": "sessions",
    "id": "sess_89324792184",
    "attributes": {
      "streamUrl": "https://icecast.radio-station.com/live.mp3",
      "title": "Evening Drive Radio Show",
      "language": "en",
      "status": "completed",
      "createdAt": "2026-06-03T07:15:00Z"
    },
    "relationships": {
      "transcripts": {
        "links": {
          "self": "https://api.transcriptions.io/api/v1/sessions/sess_893247921",
          "related": "https://api.transcriptions.io/api/v1/sessions/sess_893247"
        }
      }
    },
    "links": {
      "self": "https://api.transcriptions.io/api/v1/sessions/sess_89324792184"
    }
  }
],
"meta": {
  "totalRecords": 142,
  "totalPages": 142
}
}

```

Endpoint 4: Fetch Specific Session Details

- **Method:** GET
- **Path:** /api/v1/sessions/{id}
- **Description:** Fetches individual status parameters and relationships of a session job.

Response Document (200 OK)

```

{
  "jsonapi": {
    "version": "1.1"
  },
  "links": {
    "self": "https://api.transcriptions.io/api/v1/sessions/sess_89324792184"
  },
  "data": {
    "type": "sessions",
    "id": "sess_89324792184",
    "attributes": {
      "streamUrl": "https://icecast.radio-station.com/live.mp3",
      "title": "Evening Drive Radio Show",
      "language": "en",
      "status": "completed",
      "durationSeconds": 3600,
      "audioQualityScore": 0.94,
      "createdAt": "2026-06-03T07:15:00Z"
    }
  }
}

```

```

    },
    "relationships": {
      "transcripts": {
        "links": {
          "self": "https://api.transcriptions.io/api/v1/sessions/sess_89324792184",
          "related": "https://api.transcriptions.io/api/v1/sessions/sess_89324792"
        }
      }
    }
  }
}

```

Endpoint 5: Get Supported Model Configurations

- **Method:** GET
- **Path:** /api/v1/languages
- **Description:** Returns all language parameters, dynamic processing pipelines, and localized AI engine models enabled on the server.

Response Document (200 OK)

```

{
  "jsonapi": {
    "version": "1.1"
  },
  "links": {
    "self": "https://api.transcriptions.io/api/v1/languages"
  },
  "data": [
    {
      "type": "languages",
      "id": "en",
      "attributes": {
        "name": "English",
        "direction": "ltr",
        "modelsAvailable": ["english-base", "english-large", "realtime-low-latency"],
        "isMultilingualEngine": false,
        "features": {
          "punctuation": true,
          "speakerDiarization": true,
          "translation": false
        }
      }
    },
    {
      "type": "languages",
      "id": "es",
      "attributes": {
        "name": "Spanish",
        "direction": "ltr",
        "modelsAvailable": ["multilingual-large", "realtime-low-latency"],
        "isMultilingualEngine": true,
        "features": {
          "punctuation": true,
          "speakerDiarization": false,
          "translation": true
        }
      }
    }
  ]
}

```

```
    }
  }
],
"meta": {
  "currentSystemModelCount": 14,
  "systemOverloaded": false
}
}
```

4. WebSockets Transportation Layer

Our real-time transport maps the JSON:API design principles onto structured WebSocket event frames to maintain consistency throughout the application.

4.1 Frame Payload Wrappers (JSON:API style)

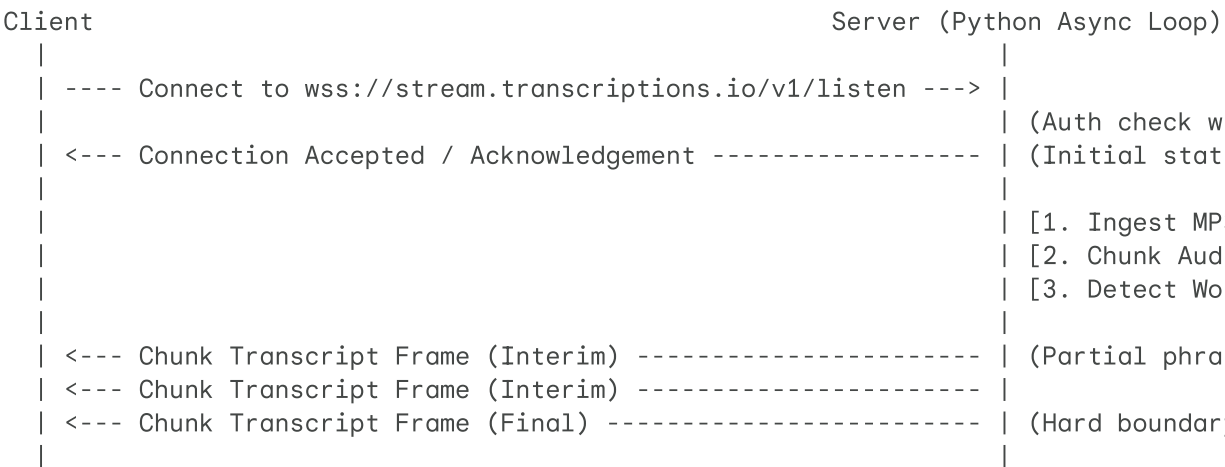
All WebSocket packets are sent as serialized JSON frames unless marked explicitly as raw audio binary. The system enforces standard parameters:

- event : Identifies the schema payload structure.
- jsonapi : Identifies compliance version.
- data : Contains structural payload metadata.

4.2 Stream Pipeline Workflows

Scenario A: Server Pull (Direct Stream URL)

When a server processes the URL itself, the client opens a WebSocket connection to *receive* continuous text segments.



Scenario B: Client Push (Mobile App Streams Audio)

The mobile app handles encoding (e.g. recording a microphone/line-in feed) and streams audio bytes over a secure socket.




```

| ---- Connection Handshake with HandshakeToken -----> |
| <---- Connection Upgraded -----> | (Ready for St
|
| ---- Binary Packet: Raw PCM bytes (16kHz, 16bit, Mono) -----> |
| ---- Binary Packet: Raw PCM bytes (16kHz, 16bit, Mono) -----> |
| ---- Binary Packet: Raw PCM bytes (16kHz, 16bit, Mono) -----> |
|
|
| [1. Process a
| [2. Run Infer
|
| <---- Chunk Transcript Frame (Interim text update) -----> |
|
| ---- Binary Packet: Raw PCM bytes -----> |
|
| <---- Chunk Transcript Frame (Final text update) -----> |
|

```

4.3 WebSocket Message Frame Structures

Frame 1: Mobile App Audio Packet (Client-to-Server Binary Data)

- **Payload Type:** Binary Frame (bytes)
- **Format:** Raw PCM (Uncompressed Signed 16-bit, Little Endian, Single-Channel 16000Hz).
- **Buffer Size:** Sends 100 ms to 250 ms packet windows (e.g., 3200 bytes to 8000 bytes per frame at 16kHz Mono).

Frame 2: Live Transcript Stream (Server-to-Client JSON Document)

The server broadcasts transcription updates down to connected listeners.

```

{
  "jsonapi": {
    "version": "1.1"
  },
  "event": "transcript_update",
  "data": {
    "type": "transcript-chunks",
    "id": "chnk_17498147",
    "attributes": {
      "sessionId": "sess_89324792184",
      "text": "the local broadcast has finished preparing dynamic audio blocks",
      "isFinal": false,
      "startTime": 12.42,
      "endTime": 15.89,
      "confidence": 0.985,
      "language": "en",
      "speakerId": "speaker_A"
    }
  }
}

```

- **Attributes Breakdown:**

- isFinal :

- `false` (Interim): Emitted instantly as the local neural net continuously processes audio slices. Text value might morph as linguistic context matures.
- `true` (Final): Emitted when a semantic boundary (such as dynamic silence / Voice Activity Detection (VAD)) is crossed. The word blocks are locked.
- `confidence` : Arithmetic average of local token probability metrics ($\in [0.0, 1.0]$).

Frame 3: Processing Error Payload (Server-to-Client JSON Document)

For streaming interruptions or model dropouts.

```
{
  "jsonapi": {
    "version": "1.1"
  },
  "event": "transcription_error",
  "errors": [
    {
      "status": "422",
      "code": "MEDIA_DECODING_FAILED",
      "title": "Stream Source Read Failure",
      "detail": "Failed to read the payload segment. The external radio stream cl",
      "source": {
        "pointer": "/data/attributes/streamUrl"
      },
      "meta": {
        "retryAllowed": true,
        "timestamp": "2026-06-03T07:20:11Z"
      }
    }
  ]
}
```